Fixing the igb Driver XDP/AF_XDP TX Queue Stall

A Deep Dive into NAPI Polling, `napi_disable()`, and the Redundant `napi_synchronize()` in the Linux Kernel

Alex Dvoretsky

March 2026

Based on a single-line patch for the Linux kernel `igb` driver targeting the Intel I210/I211 Gigabit Ethernet controllers.

Kernel version: 6.17 | Patch version: v2

Contents

1 Introduction **1**
- 1.1 The Problem . . . 1
- 1.2 Scope of This Book . . . 1
- 1.3 Affected Hardware . . . 2

2 NAPI Polling Architecture **3**
- 2.1 The Interrupt-to-Poll Transition . . . 3
- 2.2 The `napi_struct` . . . 4
- 2.3 The Budget Contract . . . 4
- 2.4 `napi_complete_done()`: The Completion CAS Loop . . . 4
- 2.5 `napi_synchronize()`: The Passive Wait . . . 5
  - 2.5.1 Historical Context . . . 6
- 2.6 `napi_disable()`: The Active Shutdown . . . 6
- 2.7 `__napi_poll()`: The Forced Completion Mechanism . . . 7
- 2.8 `igb_poll()`: The igb Driver's NAPI Callback . . . 8
- 2.9 The `__IGB_DOWN` Early-Exit Pattern . . . 9

3 AF_XDP Zero-Copy in igb **10**
- 3.1 AF_XDP Architecture . . . 10
- 3.2 Zero-Copy vs. Copy Mode . . . 10
- 3.3 The XSK Buffer Pool . . . 11
- 3.4 `igb_clean_rx_irq_zc()`: The Zero-Copy RX Path . . . 11
  - 3.4.1 The Return Value Semantics . . . 12
  - 3.4.2 What Happens When the Pool Is Destroyed . . . 12
- 3.5 Buffer Pool Lifecycle . . . 13

4 XDP Program Lifecycle and Device Reset **14**
- 4.1 `igb_xdp_setup()`: Installing and Removing XDP Programs . . . 14
- 4.2 `igb_close()`: Tearing Down the Device . . . 15
- 4.3 `igb_down()`: The Shutdown Sequence . . . 15
- 4.4 `igb_up()`: Bringing the Device Up . . . 16

5 The TX Watchdog: A Symptom, Not the Cause **18**
- 5.1 `dev_watchdog()`: The Network TX Watchdog Timer . . . 18
- 5.2 Why the Watchdog Fires During the Bug . . . 19

6 The Fix: Removing `napi_synchronize()` **20**
- 6.1 The Patch . . . 20
- 6.2 Commit Message . . . 20
- 6.3 Why `napi_synchronize()` Is Redundant . . . 21
- 6.4 Why The Fix Is Correct . . . 21

6.5 What About `__IGB_DOWN` Checks? . . . 21
6.6 The Full Failure Sequence—Before and After . . . 22
6.6.1 Before the Fix . . . 22
6.6.2 After the Fix . . . 22

7 From v1 to v2: Evolution of the Fix **23**
7.1 The v1 Series: Defense in Depth . . . 23
7.1.1 Patch 1: `__IGB_DOWN` Check in `igb_clean_rx_irq_zc()` . . . 23
7.1.2 Patch 2: `__IGB_DOWN` Check in `igb_tx_timeout()` . . . 23
7.1.3 Patch 3: XDP Transition Guards in `igb_xdp_setup()` . . . 24
7.2 Maciej Fijalkowski's Insight . . . 24
7.3 Lessons Learned . . . 24

A Full Patch Listing **26**
A.1 `[PATCH net v2] igb: remove napi_synchronize() in igb_down()` . . . 26

B How to Apply and Test the Patch **28**
B.1 Prerequisites . . . 28
B.2 Applying the Patch . . . 28
B.3 Building and Installing . . . 28
B.4 Testing Procedure . . . 28
B.4.1 Test 1: Kill AF_XDP Application . . . 28
B.4.2 Test 2: Remove XDP Program While AF_XDP Is Running . . . 29
B.4.3 Test 3: Kill-9 Stress Test (5 Cycles) . . . 29
B.5 Expected Results . . . 29

C How to Submit Patches to the Linux Kernel Mailing List **30**
C.1 Formatting . . . 30
C.2 Running `checkpatch.pl` . . . 30
C.3 Identifying the Maintainers . . . 30
C.4 Sending with `git send-email` . . . 30
C.5 The Review Process . . . 31
C.6 Backporting to Stable . . . 31

Chapter 1

Introduction

1.1 The Problem

Consider a typical deployment scenario: an Intel I210 Gigabit Ethernet controller driven by the `igb` kernel module, with an XDP program attached and an AF_XDP zero-copy application processing packets at high speed. Everything works perfectly—until the AF_XDP application is killed.

```
$ kill -9 $AF_XDP_PID
```

Within five seconds, the kernel logs an ominous message:

```
NETDEV WATCHDOG: CPU: 3: enp8s0 (igb): transmit queue 0 timed out 5008 ms
```

The interface becomes completely unresponsive. No packets are transmitted or received. Pings fail, SSH sessions hang, and the only recovery is to reload the `igb` module or reboot the machine.

The root cause is a redundant `napi_synchronize()` call in `igb_down()`. When the AF_XDP buffer pool is destroyed, the zero-copy RX poll function enters an infinite loop, returning full budget on every cycle. `napi_synchronize()` spins waiting for this poll to finish—but it never will. The subsequent `napi_disable()`, which *can* break the loop via the `NAPI_STATE_DISABLE` mechanism, never gets a chance to run.

The fix is a single line: remove the `napi_synchronize()` call.

1.2 Scope of This Book

This book explains the Linux kernel subsystems that interact to produce this bug, then walks through the fix and why it is correct. Each chapter builds the prerequisite knowledge:

Chapter 2
: explains the NAPI polling architecture—how interrupts are coalesced into poll cycles, how poll completion works, and the critical difference between `napi_synchronize()` and `napi_disable()`.

Chapter 3
: covers AF_XDP zero-copy in the `igb` driver: the buffer pool, the ZC receive path, and what happens when the pool is destroyed.

Chapter 4
: traces the lifecycle of an XDP program: how `igb_xdp_setup()` installs and removes programs, and how `igb_close()`/`igb_open()` perform the device reset.

Chapter 5
: describes the TX watchdog timer—the visible symptom of the bug, though not its root cause.

Chapter 6
: analyzes the v2 fix: why removing `napi_synchronize()` is correct and sufficient.

Chapter 7
: traces the evolution from an initial 3-patch v1 series to the minimal v2 fix, following reviewer feedback from Intel's Maciej Fijalkowski.

1.3 Affected Hardware

The bug affects all hardware supported by the `igb` driver that has AF_XDP zero-copy support. In practice, this means the Intel I210 and I211 controllers (PCI device IDs `0x1533` and `0x1539`), which are common on embedded and server platforms.

The testing for this patch was performed on an Intel I210 (PCI address `0000:08:00.0`) using the `igb` driver version shipped with Linux 6.17.

Chapter 2

NAPI Polling Architecture

NAPI (New API) is the Linux kernel's mechanism for efficient network packet processing. Rather than handling every incoming packet via a hardware interrupt, NAPI coalesces interrupts into *poll cycles* where the driver processes packets in batches.

Understanding NAPI is essential for this book because the TX stall bug is, at its core, a NAPI poll loop that never terminates—and the fix relies on understanding how `napi_disable()` breaks such loops from the inside.

2.1 The Interrupt-to-Poll Transition

The lifecycle of a NAPI poll cycle has four phases:

1. **Hardware interrupt fires.** The NIC signals that packets are available (or TX descriptors have been completed).

2. **Driver schedules NAPI.** The interrupt handler calls `napi_schedule()`, which sets the `NAPI_STATE_SCHED` bit and adds the NAPI instance to the per-CPU poll list.

3. **Softirq runs the poll function.** The `NET_RX_SOFTIRQ` handler calls the driver's `poll` callback with a `budget` (typically 64 packets).

4. **Poll completes or re-arms.** If the driver processed fewer packets than the budget (`work_done < budget`), it calls `napi_complete_done()` to signal completion and re-enable interrupts. If it consumed the full budget, it returns `budget` and NAPI will call it again.

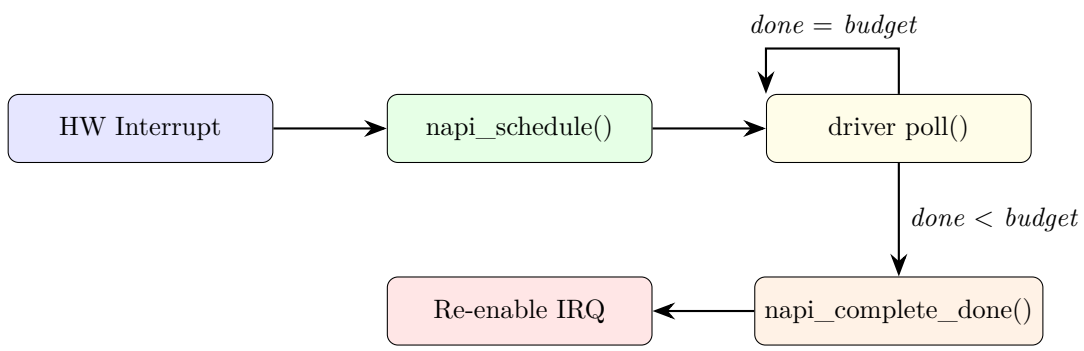

Figure 2.1: NAPI poll lifecycle. The poll function is called repeatedly as long as it returns `budget`; it exits the cycle by returning less than `budget`.

2.2 The napi_struct

Each NAPI instance is represented by a napi_struct:

```
struct napi_struct {
    unsigned long     state;
    struct list_head  poll_list;
    int               weight;
    int               (*poll)(struct napi_struct *, int);
    struct net_device *dev;
    struct hrtimer    timer;
    /* ... */
};
```

Listing 2.1: Simplified napi_struct (include/linux/netdevice.h)

The state field is a bitmask of flags. The ones relevant to this book are:

Flag	Meaning
NAPI_STATE_SCHED	NAPI is scheduled for polling. Set by napi_schedule_prep(), cleared by napi_complete_done().
NAPI_STATE_MISSED	New work arrived while NAPI was completing. If napi_complete_done() sees this flag, it re-schedules the poll instead of completing.
NAPI_STATE_DISABLE	NAPI is being disabled. Set by napi_disable(). napi_schedule_prep() will refuse to schedule when this is set. Crucially, __napi_poll() checks this flag after every full-budget poll return and forces completion if set.

Table 2.1: Key NAPI state flags.

2.3 The Budget Contract

The NAPI budget contract is the fundamental agreement between the kernel and the driver's poll function:

- The kernel calls poll(napi, budget) with a positive budget (default weight is 64).
- If the driver returns a value **equal to** budget, the kernel assumes there is more work and will call poll again.
- If the driver returns a value **less than** budget, the driver must have called napi_complete_done() to signal that the poll cycle is finished.

This contract is critical: if a poll function *always* returns budget without ever returning a smaller value, NAPI will keep polling forever. This is exactly what happens in the bug we are fixing.

2.4 napi_complete_done(): The Completion CAS Loop

napi_complete_done() is the function that transitions a NAPI instance from the "polling" state back to the "idle" state. Its implementation contains an atomic compare-and-swap (CAS) loop that handles the MISSED flag:

```c
bool napi_complete_done(struct napi_struct *n, int work_done)
{
    unsigned long flags, val, new, timeout = 0;
    bool ret = true;

    if (unlikely(n->state & (NAPIF_STATE_NPSVC |
                             NAPIF_STATE_IN_BUSY_POLL)))
        return false;

    /* ... GRO flush, poll_list removal ... */

    val = READ_ONCE(n->state);
    do {
        WARN_ON_ONCE(!(val & NAPIF_STATE_SCHED));

        new = val & ~(NAPIF_STATE_MISSED | NAPIF_STATE_SCHED |
                      NAPIF_STATE_SCHED_THREADED |
                      NAPIF_STATE_PREFER_BUSY_POLL);

        /* If MISSED was set, keep SCHED set for one more poll */
        new |= (val & NAPIF_STATE_MISSED) /
               NAPIF_STATE_MISSED * NAPIF_STATE_SCHED;
    } while (!try_cmpxchg(&n->state, &val, new));

    if (unlikely(val & NAPIF_STATE_MISSED)) {
        __napi_schedule(n);
        return false;
    }

    /* ... hrtimer for GRO ... */
    return ret;
}
```

Listing 2.2: `napi_complete_done()` core logic (net/core/dev.c)

The key insight is the `MISSED` flag re-scheduling logic. When another CPU (or the driver itself) sets `MISSED` during the CAS window, `napi_complete_done()` sees it and re-schedules NAPI instead of completing.

2.5 napi_synchronize(): The Passive Wait

`napi_synchronize()` is a simple busy-wait that spins until the `NAPI_STATE_SCHED` bit is clear:

```c
static inline void napi_synchronize(const struct napi_struct *n)
{
    if (IS_ENABLED(CONFIG_SMP))
        while (test_bit(NAPI_STATE_SCHED, &n->state))
            msleep(1);
    else
        barrier();
}
```

Listing 2.3: `napi_synchronize()` (include/linux/netdevice.h)

This function is purely passive: it waits for the poll function to finish on its own. It does *not* set any flags, does not signal the poll function, and does not prevent future scheduling. It simply polls the `NAPI_STATE_SCHED` bit with a 1 ms sleep between iterations.

Critical property: if the poll function never returns `done < budget`, then `napi_complete_done()` is never called, `NAPI_STATE_SCHED` is never cleared, and `napi_synchronize()` **spins forever**. This is not a deadlock—it is a livelock: both the poll function and the synchronize caller are making progress (burning CPU), but neither can terminate.

2.5.1 Historical Context

The `napi_synchronize()` call in `igb_down()` was introduced in commit `41f149a285da` ("igb: Fix possible panic caused by Rx traffic arrival while interface is down"). The intent was to ensure that no NAPI poll callback was actively running before disabling NAPI, so that the poll function would not access freed resources.

At the time this commit was written (2009), the NAPI subsystem's `napi_disable()` was less sophisticated. The `NAPI_STATE_DISABLE` mechanism and the `napi_disable_pending()` check in `__napi_poll()` were later refinements that made the explicit synchronize unnecessary.

2.6 `napi_disable()`: The Active Shutdown

`napi_disable()` is the intended way to shut down a NAPI instance during device teardown. Unlike `napi_synchronize()`, it actively *signals* the poll function to stop:

```
void napi_disable(struct napi_struct *n)
{
    unsigned long val, new;

    might_sleep();
    set_bit(NAPI_STATE_DISABLE, &n->state);

    val = READ_ONCE(n->state);
    do {
        while (val & NAPIF_STATE_SCHED) {
            usleep_range(20, 200);
            val = READ_ONCE(n->state);
        }

        new = val | NAPIF_STATE_SCHED |
                    NAPIF_STATE_NPSVC;
        new &= ~(NAPIF_STATE_DISABLE |
                 NAPIF_STATE_PREFER_BUSY_POLL);
    } while (!try_cmpxchg(&n->state, &val, new));
}
```

Listing 2.4: `napi_disable()` (net/core/dev.c)

The key steps are:

1. **Set `NAPI_STATE_DISABLE`.** This is the signal. Once set, the NAPI core will intervene to break any stuck poll loop (see Section 2.7).

2. **Wait for `NAPI_STATE_SCHED` to clear.** Like `napi_synchronize()`, this waits for the current poll to finish. But because `NAPI_STATE_DISABLE` is now set, the NAPI core will *force* the poll to finish (see below).

3. **Atomically set `SCHED` and `NPSVC`.** This prevents any future scheduling—the NAPI instance is now permanently disabled until `napi_enable()` is called.

The crucial difference: napi_synchronize() waits passively for SCHED to clear. napi_disable() *also* waits—but it first sets DISABLE, which triggers the forced completion mechanism in __napi_poll(). This is why napi_disable() can handle a stuck poll loop and napi_synchronize() cannot.

2.7 __napi_poll(): The Forced Completion Mechanism

The NAPI core function __napi_poll() is where the magic happens. After calling the driver's poll function, it checks whether a disable is pending:

```
static int __napi_poll(struct napi_struct *n, bool *repoll)
{
    int work, weight;

    weight = n→weight;
    work = n→poll(n, weight);

    if (likely(work < weight))
        return work;

    /* Driver returned full budget.
     * Normally we would repoll, but check for disable first. */
    if (unlikely(napi_disable_pending(n))) {
        napi_complete(n); /* forces SCHED clear */
        return work;
    }

    /* ... busy poll, time limit checks ... */

    *repoll = true;
    return work;
}
```

Listing 2.5: __napi_poll() forced completion (net/core/dev.c)

napi_disable_pending() simply checks the NAPI_STATE_DISABLE bit:

```
static inline bool napi_disable_pending(struct napi_struct *n)
{
    return test_bit(NAPI_STATE_DISABLE, &n→state);
}
```

Listing 2.6: napi_disable_pending() (include/linux/netdevice.h)

When the poll function returns full budget *and* NAPI_STATE_DISABLE is set, __napi_poll() calls napi_complete() to force-clear NAPI_STATE_SCHED. This breaks the infinite loop:

1. napi_disable() sets NAPI_STATE_DISABLE.
2. The poll function runs and returns full budget (the stuck loop).
3. __napi_poll() sees napi_disable_pending() is true.
4. napi_complete() clears NAPI_STATE_SCHED.
5. napi_disable() sees SCHED clear and proceeds to finalize the shutdown.

This mechanism was designed precisely for situations where a poll function is stuck returning full budget. It allows `napi_disable()` to reliably shut down any NAPI instance, regardless of what the poll function is doing.

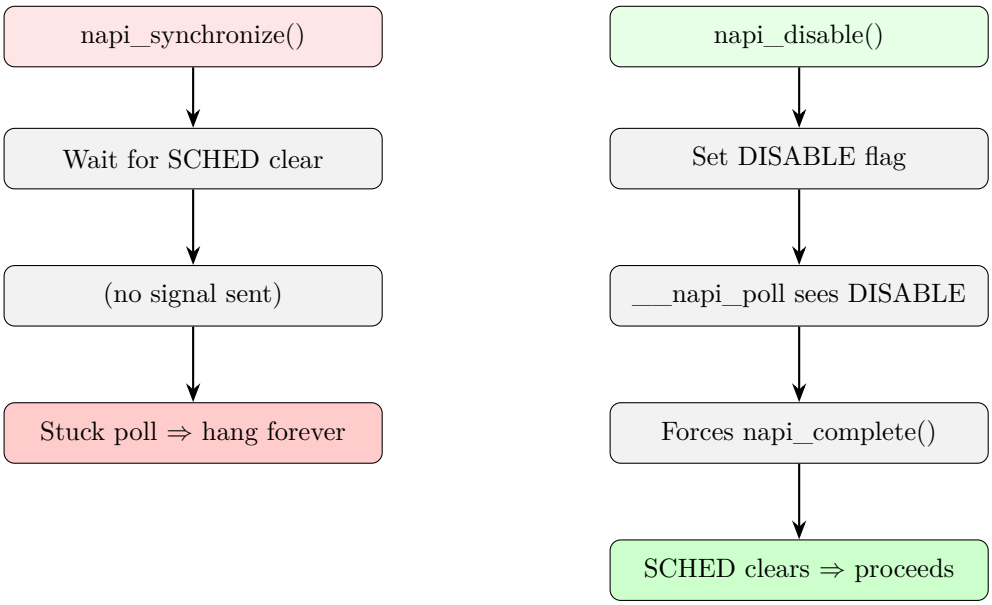

Figure 2.2: Comparison of `napi_synchronize()` vs `napi_disable()` when a poll function is stuck returning full budget. Only `napi_disable()` can break the loop.

2.8 `igb_poll()`: The igb Driver's NAPI Callback

The `igb` driver registers `igb_poll()` as the NAPI poll callback. It processes TX completions first, then RX packets:

```
static int igb_poll(struct napi_struct *napi, int budget)
{
    struct igb_q_vector *q_vector = container_of(napi,
                        struct igb_q_vector, napi);
    struct xsk_buff_pool *xsk_pool;
    bool clean_complete = true;
    int work_done = 0;

    if (q_vector->tx.ring)
        clean_complete = igb_clean_tx_irq(q_vector, budget);

    if (q_vector->rx.ring) {
        int cleaned;

        xsk_pool = READ_ONCE(q_vector->rx.ring->xsk_pool);
        cleaned = xsk_pool ?
            igb_clean_rx_irq_zc(q_vector, xsk_pool, budget) :
            igb_clean_rx_irq(q_vector, budget);

        work_done += cleaned;
        if (cleaned >= budget)
            clean_complete = false;
    }

    if (!clean_complete)
```

```
        return budget;

    if (likely(napi_complete_done(napi, work_done)))
        igb_ring_irq_enable(q_vector);

    return work_done;
}
```

Listing 2.7: `igb_poll()` (igb_main.c:8280)

The decision tree is:

1. If `igb_clean_tx_irq()` returns `false` (more TX work), `clean_complete` is set to `false`.
2. If the RX cleaner returns `budget` or more, `clean_complete` is set to `false`.
3. If `clean_complete` is `false`, return `budget` without calling `napi_complete_done()`—NAPI will call us again.
4. Otherwise, call `napi_complete_done()` and re-enable interrupts.

2.9 The __IGB_DOWN Early-Exit Pattern

`igb_clean_tx_irq()` begins with an early-exit check:

```
static bool igb_clean_tx_irq(struct igb_q_vector *q_vector,
                             int napi_budget)
{
    struct igb_adapter *adapter = q_vector→adapter;
    /* ... */

    if (test_bit(__IGB_DOWN, &adapter→state))
        return true;

    /* ... process TX completions ... */
}
```

Listing 2.8: `__IGB_DOWN` check in `igb_clean_tx_irq()` (igb_main.c:8344)

When the adapter is going down (`__IGB_DOWN` is set by `igb_down()`), `igb_clean_tx_irq()` returns `true` immediately. In the context of `igb_poll()`, `true` means "TX work is complete" (sets `clean_complete = true`).

This pattern is relevant because `igb_clean_rx_irq_zc()` *lacks* the same check—but as we will see in Chapter 6, the v2 fix makes such a check unnecessary by removing the `napi_synchronize()` that would hang.

Chapter 3

AF_XDP Zero-Copy in igb

AF_XDP is a high-performance packet I/O mechanism that allows user-space applications to send and receive network packets with minimal kernel overhead. When used in *zero-copy mode*, the NIC's DMA engine reads and writes directly to user-space memory, bypassing the kernel's `sk_buff` allocation entirely.

3.1 AF_XDP Architecture

An AF_XDP socket operates on a shared memory region called a UMEM (User Memory for Efficient Mapping). The UMEM is divided into fixed-size frames, and four ring buffers coordinate frame ownership between the kernel and user-space:

Ring	**Direction**	**Purpose**
Fill Queue (FQ)	User → Kernel	User provides empty frames for RX
Completion Queue (CQ)	Kernel → User	Kernel returns frames after TX
RX Ring	Kernel → User	Kernel delivers received packets
TX Ring	User → Kernel	User submits packets for TX

Table 3.1: AF_XDP ring buffers.

In zero-copy mode, the kernel maps the UMEM directly into the NIC's DMA address space. The driver programs the NIC's RX descriptor ring with DMA addresses pointing into the UMEM, so received packets land directly in user-space memory.

3.2 Zero-Copy vs. Copy Mode

Copy mode The kernel receives packets into its own buffers via the normal RX path, then copies the packet data into the AF_XDP socket's UMEM. This adds a `memcpy` per packet but works with any driver.

Zero-copy mode The driver is aware of AF_XDP and programs the NIC to DMA directly into the UMEM. This eliminates the copy but requires explicit driver support. The `igb` driver gained zero-copy support in commit `2c6196013f84` ("igb: Add AF_XDP zero-copy Rx support").

3.3 The XSK Buffer Pool

The kernel-side representation of the UMEM for a particular queue is the `xsk_buff_pool`. It is created when a user-space application binds an AF_XDP socket to a specific queue:

```
struct igb_ring {
    /* ... */
    struct xsk_buff_pool *xsk_pool;
    /* ... */
    union {
        struct igb_rx_buffer *rx_buffer_info;
        struct xdp_buff **rx_buffer_info_zc;
    };
    /* ... */
};
```

Listing 3.1: XSK pool reference in `igb_ring` (igb.h)

When the pool is present, the ring uses `rx_buffer_info_zc` (an array of `xdp_buff` pointers) instead of the normal `rx_buffer_info`. Buffer allocation uses `xsk_buff_alloc()` or `xsk_buff_alloc_batch()` to obtain frames from the Fill Queue.

Key observation: when the AF_XDP application dies, the pool is destroyed. After that point, `xsk_buff_alloc()` returns `NULL`—there are no more frames to allocate.

3.4 `igb_clean_rx_irq_zc()`: The Zero-Copy RX Path

`igb_clean_rx_irq_zc()` is the RX poll function used when a ZC pool is active on a ring. It is selected by `igb_poll()` based on the presence of `xsk_pool`:

```
xsk_pool = READ_ONCE(q_vector→rx.ring→xsk_pool);
cleaned = xsk_pool ?
    igb_clean_rx_irq_zc(q_vector, xsk_pool, budget) :
    igb_clean_rx_irq(q_vector, budget);
```

Listing 3.2: ZC RX path selection in `igb_poll()` (igb_main.c:8300)

The function processes received packets from the descriptor ring, runs them through the XDP program, and allocates replacement buffers from the pool:

```
int igb_clean_rx_irq_zc(struct igb_q_vector *q_vector,
                        struct xsk_buff_pool *xsk_pool,
                        const int budget)
{
    struct igb_adapter *adapter = q_vector→adapter;
    struct igb_ring *rx_ring = q_vector→rx.ring;
    unsigned int total_packets = 0;
    /* ... */

    xdp_prog = READ_ONCE(rx_ring→xdp_prog);

    while (likely(total_packets < budget)) {
        /* Try to read a completed RX descriptor */
        rx_desc = IGB_RX_DESC(rx_ring, ntc);
        size = le16_to_cpu(rx_desc→wb.upper.length);
        if (!size)
            break;    /* no more completed descriptors */

        /* ... process the packet via XDP ... */
```

```
        total_packets++;
    }

    /* ... update ring pointers ... */

    /* Try to refill the ring with new buffers */
    entries_to_alloc = igb_desc_unused(rx_ring);
    if (entries_to_alloc >= IGB_RX_BUFFER_WRITE)
        failure |= !igb_alloc_rx_buffers_zc(rx_ring, xsk_pool,
                                            entries_to_alloc);

    if (xsk_uses_need_wakeup(xsk_pool)) {
        /* ... wakeup handling ... */
        return (int)total_packets;
    }
    return failure ? budget : (int)total_packets;
}
```

Listing 3.3: `igb_clean_rx_irq_zc()` (igb_xsk.c:341), simplified

3.4.1 The Return Value Semantics

The return value of `igb_clean_rx_irq_zc()` determines whether NAPI considers the poll complete:

- If `total_packets < budget`, NAPI calls `napi_complete_done()` and the poll cycle ends.
- If `total_packets >= budget` (or `failure` returns `budget`), NAPI continues polling.

The `failure` flag is set when buffer allocation fails—meaning the Fill Queue is empty. In that case, the function returns `budget` to request another poll cycle, hoping that the user-space application will refill the Fill Queue.

3.4.2 What Happens When the Pool Is Destroyed

When the AF_XDP application dies:

1. The XSK buffer pool is destroyed.
2. `xsk_buff_alloc_batch()` returns 0 buffers—the Fill Queue is gone.
3. `igb_alloc_rx_buffers_zc()` returns `false` (failure), so `failure = true`.
4. There are no completed RX descriptors (the NIC has no buffers to write to), so the `while` loop processes 0 packets.
5. `total_packets = 0`, but `failure = true`, so the function returns `budget`.
6. Back in `igb_poll()`: `cleaned >= budget` ⇒ `clean_complete = false` ⇒ return `budget` without calling `napi_complete_done()`.
7. NAPI re-schedules the poll. Go to step 2.

This creates an **infinite NAPI poll loop**: the function keeps returning `budget` because it cannot allocate buffers, and NAPI keeps calling it because it thinks there is more work to do.

This infinite loop is the trigger condition for the bug. However, as we will see, the infinite loop *alone* does not cause the TX stall. The problem is that `igb_down()` calls `napi_synchronize()` *before* `napi_disable()`. `napi_synchronize()` cannot break the loop; `napi_disable()` can. By the time `napi_disable()` would run, `igb_down()` is already stuck.

3.5 Buffer Pool Lifecycle

The full lifecycle of a zero-copy buffer pool:

1. **Creation.** User-space creates an AF_XDP socket and binds it to queue N. The kernel creates an `xsk_buff_pool` and stores it in `adapter->rx_ring[N]->xsk_pool`.
2. **Active use.** The driver's ZC RX/TX paths use the pool to allocate and complete buffers.
3. **Destruction.** The AF_XDP socket is closed (either normally or because the process dies). The pool is destroyed, and `xsk_pool` is set to `NULL`—but not necessarily before the NAPI poll function has already read the old pointer.

The race between pool destruction and NAPI polling is what triggers the infinite loop: `igb_poll()` reads a non-NULL `xsk_pool` pointer and calls `igb_clean_rx_irq_zc()`, but by the time the function tries to allocate buffers, the pool is already gone.

Chapter 4

XDP Program Lifecycle and Device Reset

4.1 igb_xdp_setup(): Installing and Removing XDP Programs

The netdev core calls `igb_xdp_setup()` when an XDP program is attached to or detached from the interface. The function handles two cases:

Program replacement (e.g., updating the XDP program without changing modes): the new program is atomically swapped in without a reset.

Mode transition (XDP → non-XDP or non-XDP → XDP): the device must be reset because the ring layout changes.

```
static int igb_xdp_setup(struct net_device *dev,
                         struct netdev_bpf *bpf)
{
    struct igb_adapter *adapter = netdev_priv(dev);
    struct bpf_prog *prog = bpf->prog, *old_prog;
    bool running = netif_running(dev);
    bool need_reset;

    /* ... frame size validation ... */

    old_prog = xchg(&adapter->xdp_prog, prog);
    need_reset = (!!prog != !!old_prog);

    if (need_reset && running) {
        igb_close(dev);
    } else {
        for (i = 0; i < adapter->num_rx_queues; i++)
            (void)xchg(&adapter->rx_ring[i]->xdp_prog,
                adapter->xdp_prog);
    }

    if (old_prog)
        bpf_prog_put(old_prog);

    if (!need_reset)
        return 0;

    /* ... xdp_features update ... */
```

```

    if (running)
        igb_open(dev);

    return 0;
}
```

Listing 4.1: igb_xdp_setup() (igb_main.c:2890)

When a mode transition occurs on a running device, the function calls igb_close() followed by igb_open(). This is a complete device teardown and reinitialization—the most disruptive operation the driver performs.

4.2 igb_close(): Tearing Down the Device

igb_close() is the network device's close callback (ndo_stop). It delegates to __igb_close():

```
static int __igb_close(struct net_device *netdev,
                       bool suspending)
{
    struct igb_adapter *adapter = netdev_priv(netdev);

    igb_down(adapter);
    igb_free_irq(adapter);
    igb_free_all_tx_resources(adapter);
    igb_free_all_rx_resources(adapter);
    /* ... */
    return 0;
}
```

Listing 4.2: igb_close() and __igb_close() (igb_main.c:4259)

The real work happens in igb_down().

4.3 igb_down(): The Shutdown Sequence

igb_down() is the core shutdown function. Understanding its sequence is essential for understanding the bug:

```
void igb_down(struct igb_adapter *adapter)
{
    struct net_device *netdev = adapter->netdev;
    struct e1000_hw *hw = &adapter->hw;

    /* (1) Signal shutdown */
    set_bit(__IGB_DOWN, &adapter->state);

    /* (2) Disable HW RX */
    rctl = rd32(E1000_RCTL);
    wr32(E1000_RCTL, rctl & ~E1000_RCTL_EN);

    /* (3) Stop TX queues, disable HW TX */
    netif_carrier_off(netdev);
    netif_tx_stop_all_queues(netdev);
    tctl = rd32(E1000_TCTL);
    tctl &= ~E1000_TCTL_EN;
    wr32(E1000_TCTL, tctl);
    wrfl();
```

```
20     usleep_range(10000, 11000);
21
22     /* (4) Disable interrupts */
23     igb_irq_disable(adapter);
24
25     /* (5) Wait for NAPI, then disable it */
26     for (i = 0; i < adapter->num_q_vectors; i++) {
27         if (adapter->q_vector[i]) {
28             napi_synchronize(        /* <-- THE BUG */
29                 &adapter->q_vector[i]->napi);
30             igb_set_queue_napi(adapter, i, NULL);
31             napi_disable(
32                 &adapter->q_vector[i]->napi);
33         }
34     }
35
36     /* (6) Cancel timers, update stats, reset HW */
37     timer_delete_sync(&adapter->watchdog_timer);
38     /* ... */
39     igb_reset(adapter);
40
41     /* (7) Clean all rings */
42     igb_clean_all_tx_rings(adapter);
43     igb_clean_all_rx_rings(adapter);
44 }
```

Listing 4.3: `igb_down()` (igb_main.c:2170), before the fix

The sequence has a clear dependency chain:

1. **Set `__IGB_DOWN`** (step 1) to signal all paths that the adapter is going down.
2. **Disable hardware** (steps 2–4) to stop new packets and interrupts.
3. **Wait for NAPI** (step 5): `napi_synchronize()` blocks until the current poll cycle completes, then `napi_disable()` prevents future scheduling.
4. **Clean up** (steps 6–7) now that no concurrent access is possible.

The bug is in step 5. `napi_synchronize()` is called *before* `napi_disable()`. If the poll function is stuck in an infinite loop (as with the ZC bug described in Section 3.4), `napi_synchronize()` hangs forever. `napi_disable()`—which *could* break the loop—never gets a chance to run.

Between the `napi_synchronize()` and `napi_disable()` calls sits `igb_set_queue_napi()`, which performs an atomic pointer update (`rxq->napi = NULL`, `txq->napi = NULL`). The poll path does not read these pointers, so this operation is safe to execute after removing `napi_synchronize()` and before `napi_disable()`.

4.4 `igb_up()`: Bringing the Device Up

`igb_up()` reverses the shutdown:

```
1 int igb_up(struct igb_adapter *adapter)
2 {
3     /* (1) Configure hardware */
4     igb_configure(adapter);
5
```

```
    /* (2) Clear shutdown flag */
    clear_bit(__IGB_DOWN, &adapter→state);

    /* (3) Enable NAPI */
    for (i = 0; i < adapter→num_q_vectors; i++) {
        napi = &adapter→q_vector[i]→napi;
        napi_enable(napi);
    }

    /* (4) Enable interrupts and start TX queues */
    igb_irq_enable(adapter);
    netif_tx_start_all_queues(adapter→netdev);

    /* (5) Start watchdog */
    schedule_work(&adapter→watchdog_task);

    return 0;
}
```

Listing 4.4: `igb_up()` key steps (igb_main.c:2122)

Chapter 5

The TX Watchdog: A Symptom, Not the Cause

The TX watchdog timer and its timeout handler are the *visible symptom* of the bug—the "transmit queue 0 timed out" message that alerts the administrator. Understanding the watchdog helps explain *why* the bug manifests as a TX stall, even though the root cause is in the NAPI shutdown path.

5.1 `dev_watchdog()`: The Network TX Watchdog Timer

The Linux kernel maintains a per-device watchdog timer that monitors TX queue health. The timer callback, `dev_watchdog()`, fires periodically and checks whether any *stopped* TX queue has exceeded its timeout:

```
static void dev_watchdog(struct timer_list *t)
{
    struct net_device *dev = /* ... */;

    if (netif_device_present(dev) &&
        netif_running(dev) &&
        netif_carrier_ok(dev)) {

        for (i = 0; i < dev->num_tx_queues; i++) {
            txq = netdev_get_tx_queue(dev, i);
            if (!netif_xmit_stopped(txq))
                continue;

            trans_start = READ_ONCE(txq->trans_start);

            if (time_after(jiffies,
                    trans_start + dev->watchdog_timeo)) {
                /* TIMEOUT! */
                netdev_crit(dev,
                    "NETDEV WATCHDOG: CPU: %d: "
                    "transmit queue %u timed out "
                    "%u ms\n", ...);
                dev->netdev_ops->ndo_tx_timeout(dev, i);
            }
        }
    }
}
```

Listing 5.1: `dev_watchdog()` (net/sched/sch_generic.c:500), simplified

The default `watchdog_timeo` is 5 seconds (`5*HZ`).

5.2 Why the Watchdog Fires During the Bug

When `igb_down()` hangs at `napi_synchronize()`:

1. The hardware TX and RX are already disabled (steps 2–3 of `igb_down()`).
2. TX queues are stopped (`netif_tx_stop_all_queues()`).
3. The `trans_start` timestamps are stale—they reflect the last transmission *before* the shutdown began.
4. After 5 seconds, `dev_watchdog()` sees stopped queues with stale timestamps and fires `igb_tx_timeout()`.
5. `igb_tx_timeout()` schedules `igb_reset_task()`, but the reset cannot proceed because `igb_down()` is stuck.
6. The TX queue remains permanently stalled.

With the v2 fix, `igb_down()` completes quickly (via `napi_disable()`'s forced completion), so the watchdog never sees a 5-second gap. The entire watchdog-timeout-reset chain never triggers.

Chapter 6

The Fix: Removing napi_synchronize()

6.1 The Patch

The v2 fix is a single-line deletion:

```
for (i = 0; i < adapter→num_q_vectors; i++) {
    if (adapter→q_vector[i]) {
        napi_synchronize(&adapter→q_vector[i]→napi);
        igb_set_queue_napi(adapter, i, NULL);
        napi_disable(&adapter→q_vector[i]→napi);
    }
}
```

Listing 6.1: The v2 patch (igb_main.c)

6.2 Commit Message

```
igb: remove napi_synchronize() in igb_down()

When an AF_XDP zero-copy application terminates abruptly (e.g.,
kill -9), the XSK buffer pool is destroyed but NAPI polling
continues. igb_clean_rx_irq_zc() repeatedly returns the full
budget, preventing napi_complete_done() from clearing
NAPI_STATE_SCHED.

igb_down() calls napi_synchronize() before napi_disable() for
each queue vector. napi_synchronize() spins waiting for
NAPI_STATE_SCHED to clear, which never happens. igb_down()
blocks indefinitely, the TX watchdog fires, and the TX queue
remains permanently stalled.

napi_disable() already handles this correctly: it sets
NAPI_STATE_DISABLE. After a full-budget poll, __napi_poll()
checks napi_disable_pending(). If set, it forces completion
and clears NAPI_STATE_SCHED, breaking the loop that
napi_synchronize() cannot.

napi_synchronize() was added in commit 41f149a285da ("igb: Fix
possible panic caused by Rx traffic arrival while interface is
down"). napi_disable() provides stronger guarantees: it prevents
further scheduling and waits for any active poll to exit.
Other Intel drivers (ixgbe, ice, i40e) use napi_disable()
without a preceding napi_synchronize() in their down paths.
```

```
Remove redundant napi_synchronize() call.

Fixes: 2c6196013f84 ("igb: Add AF_XDP zero-copy Rx support")
```

6.3 Why napi_synchronize() Is Redundant

The original purpose of `napi_synchronize()` was to ensure that no NAPI poll callback was actively running before the driver proceeded with resource teardown. But `napi_disable()` already provides this guarantee and more:

Property	napi_synchronize()	napi_disable()
Waits for current poll to finish	Yes	Yes
Signals poll to stop	No	Yes (DISABLE flag)
Prevents future scheduling	No	Yes (sets SCHED+NPSVC)
Handles stuck full-budget polls	No	Yes (forced completion)

Table 6.1: Comparison of synchronization guarantees.

The synchronization that `napi_synchronize()` provides is a strict subset of what `napi_disable()` provides. Calling both in sequence is therefore redundant—`napi_disable()` alone is sufficient.

6.4 Why The Fix Is Correct

1. **No lost synchronization.** The only purpose of `napi_synchronize()` was to wait for the current poll to finish. `napi_disable()` does the same thing, but with the added ability to break stuck polls via `napi_disable_pending()`.

2. **Safe ordering of `igb_set_queue_napi()`.** Between the removed `napi_synchronize()` and `napi_disable()`, `igb_set_queue_napi()` performs atomic pointer updates (`rxq->napi = NULL`, `txq->napi = NULL`). The poll path does not read these pointers; the real resource cleanup (`igb_clean_all_tx_rings()`, `igb_clean_all_rx_rings()`) happens after all `napi_disable()` calls complete.

3. **Breaks the infinite loop.** `napi_disable()` sets `NAPI_STATE_DISABLE`. `__napi_poll()` checks `napi_disable_pending()` after the poll function returns full budget, and calls `napi_complete()` to force-clear `NAPI_STATE_SCHED`. This breaks the infinite loop that `napi_synchronize()` cannot.

4. **Cross-driver precedent.** Other Intel drivers—`ixgbe`, `ice`, `i40e`—use `napi_disable()` without a preceding `napi_synchronize()` in their down paths. Only `igb` and `igc` carry this pattern.

5. **No hot-path changes.** The fix touches only the shutdown path (`igb_down()`), not the data-plane poll functions. There is zero performance impact on normal packet processing.

6. **No new code.** The fix is a deletion, not an addition. It removes a redundant call, reducing complexity. There are no new code paths to test or maintain.

6.5 What About `__IGB_DOWN` Checks?

In the v1 patch series, the primary fix was to add an `__IGB_DOWN` check at the top of `igb_clean_rx_irq_zc()`, causing it to return 0 during shutdown and thus breaking the poll

loop. This approach worked but had a drawback: it added a conditional branch to the hot path of every ZC RX poll call.

The v2 approach is superior because it fixes the problem in the shutdown path itself. The poll function's behavior—returning `budget` when the buffer pool is destroyed—is actually valid NAPI behavior. The poll function is saying "I could do more work if given the chance." The real bug is that `igb_down()` relies on `napi_synchronize()`, which cannot handle this case.

With `napi_synchronize()` removed, `napi_disable()` handles the stuck poll correctly via its `NAPI_STATE_DISABLE` mechanism. No hot-path changes are needed.

6.6 The Full Failure Sequence—Before and After

6.6.1 Before the Fix

1. AF_XDP application is killed.
2. XSK buffer pool is destroyed.
3. `igb_clean_rx_irq_zc()` enters infinite loop (returns `budget` every cycle).
4. `igb_xdp_setup()` calls `igb_close()` → `igb_down()`.
5. `igb_down()` sets `__IGB_DOWN`, disables HW.
6. `igb_down()` calls `napi_synchronize()`—**hangs forever** (poll never returns `done < budget`).
7. 5 seconds later: TX watchdog fires, `igb_tx_timeout()` schedules reset.
8. Reset cannot proceed: `igb_down()` is stuck.
9. TX queue permanently stalled.

6.6.2 After the Fix

1. AF_XDP application is killed.
2. XSK buffer pool is destroyed.
3. `igb_clean_rx_irq_zc()` enters infinite loop.
4. `igb_xdp_setup()` calls `igb_close()` → `igb_down()`.
5. `igb_down()` sets `__IGB_DOWN`, disables HW.
6. `igb_down()` calls `napi_disable()`, which sets `NAPI_STATE_DISABLE`.
7. Next poll returns full budget; `__napi_poll()` sees `napi_disable_pending()` and calls `napi_complete()`.
8. `NAPI_STATE_SCHED` clears; `napi_disable()` proceeds.
9. `igb_down()` continues with cleanup.
10. `igb_open()` reinitializes the device.
11. Interface is back up. Total disruption: milliseconds.

Chapter 7

From v1 to v2: Evolution of the Fix

This chapter documents the evolution of the patch from the original 3-patch v1 series to the minimal v2 single-line fix. It is included for two reasons: to show how kernel patch review works in practice, and because the v1 analysis remains a valid (if more complex) fix for the same bug.

7.1 The v1 Series: Defense in Depth

The initial v1 submission consisted of three patches that addressed different phases of the failure cascade:

7.1.1 Patch 1: `__IGB_DOWN` Check in `igb_clean_rx_irq_zc()`

Added an early-exit check to the ZC RX poll function:

```
int igb_clean_rx_irq_zc(struct igb_q_vector *q_vector,
                        struct xsk_buff_pool *xsk_pool,
                        const int budget)
{
    struct igb_adapter *adapter = q_vector→adapter;
    /* ... */

   if (test_bit(__IGB_DOWN, &adapter→state))
       return 0;

    /* xdp_prog cannot be NULL in the ZC path */
    xdp_prog = READ_ONCE(rx_ring→xdp_prog);
```

This was the primary fix: when `__IGB_DOWN` is set, return 0 instead of `budget`, allowing `napi_complete_done()` to be called and unblocking `napi_synchronize()`.

7.1.2 Patch 2: `__IGB_DOWN` Check in `igb_tx_timeout()`

Added a guard to prevent spurious reset scheduling during shutdown:

```
static void igb_tx_timeout(struct net_device *netdev,
             unsigned int __always_unused txqueue)
{
    struct igb_adapter *adapter = netdev_priv(netdev);
    struct e1000_hw *hw = &adapter→hw;

   if (test_bit(__IGB_DOWN, &adapter→state))
       return;

```

```
    /* Do the reset outside of interrupt context */
    adapter->tx_timeout_count++;
```

This prevented the TX watchdog from scheduling a reset during the `igb_close()`/`igb_open()` transition window, where stale `trans_start` timestamps would cause false timeouts.

7.1.3 Patch 3: XDP Transition Guards in `igb_xdp_setup()`

Three sub-fixes in the XDP setup path:

- `synchronize_rcu()` before `igb_close()` to drain in-flight `igb_xsk_wakeup()` calls.
- `netif_trans_update()` after `igb_open()` to refresh stale `trans_start` timestamps.
- `cancel_work()` to drain any stale `reset_task()` work items queued during the transition.

7.2 Maciej Fijalkowski's Insight

After v1 was submitted to the intel-wired-lan mailing list, Intel engineer Maciej Fijalkowski reviewed the series and suggested a fundamentally different approach.

His key observation: the v1 patches were treating the symptoms (stuck poll loop, spurious watchdog, race conditions) rather than the root cause. The real problem was the redundant `napi_synchronize()` call that runs before `napi_disable()`. Since `napi_disable()` already provides all the synchronization guarantees of `napi_synchronize()` plus the ability to break stuck polls, the `napi_synchronize()` call is not just redundant—it is actively harmful in the AF_XDP teardown case.

Removing `napi_synchronize()` makes all three v1 patches unnecessary:

v1 Patch	Why v2 Makes It Unnecessary
`__IGB_DOWN` in ZC RX	Without `napi_synchronize()`, `igb_down()` never hangs. The stuck poll loop is broken by `napi_disable()` via `NAPI_STATE_DISABLE`.
`__IGB_DOWN` in tx_timeout	Since `igb_down()` completes quickly, the TX watchdog never fires. No spurious reset to suppress.
XDP transition guards	The close/open window is now short enough that the stale timestamp problem and reset-task race do not manifest.

Table 7.1: Why v2 supersedes all three v1 patches.

7.3 Lessons Learned

1. **Look for the root cause, not the symptoms.** The v1 series was a correct fix but addressed three symptoms of a single root cause. The v2 fix addresses the root cause directly.
2. **Understand the framework before patching the driver.** The key insight—that `napi_disable()` handles stuck polls via `napi_disable_pending()`—required understanding the NAPI core in `net/core/dev.c`, not just the igb driver.
3. **Fewer lines of change is almost always better.** The v1 series added 25 lines of new code across two files. The v2 fix removes 1 line. Simpler patches are easier to review, less likely to introduce regressions, and easier to backport to stable kernels.

4. **Cross-driver comparison is revealing.** Observing that `ixgbe`, `ice`, and `i40e` all use `napi_disable()` without `napi_synchronize()` confirmed that the `napi_synchronize()` call was not load-bearing.

Appendix A

Full Patch Listing

This appendix contains the verbatim v2 patch as submitted to the Linux kernel mailing list.

A.1 [PATCH net v2] igb: remove napi_synchronize() in igb_down()

```
From: Alex Dvoretsky <advoretsky@gmail.com>
Date: Tue, 11 Mar 2026 19:00:00 +0100
Subject: [PATCH net v2] igb: remove napi_synchronize() in igb_down()

When an AF_XDP zero-copy application terminates abruptly (e.g., kill -9),
the XSK buffer pool is destroyed but NAPI polling continues.
igb_clean_rx_irq_zc() repeatedly returns the full budget, preventing
napi_complete_done() from clearing NAPI_STATE_SCHED.

igb_down() calls napi_synchronize() before napi_disable() for each queue
vector. napi_synchronize() spins waiting for NAPI_STATE_SCHED to clear,
which never happens. igb_down() blocks indefinitely, the TX watchdog
fires, and the TX queue remains permanently stalled.

napi_disable() already handles this correctly: it sets NAPI_STATE_DISABLE.
After a full-budget poll, __napi_poll() checks napi_disable_pending(). If
set, it forces completion and clears NAPI_STATE_SCHED, breaking the loop
that napi_synchronize() cannot.

napi_synchronize() was added in commit 41f149a285da ("igb: Fix possible
panic caused by Rx traffic arrival while interface is down").
napi_disable() provides stronger guarantees: it prevents further
scheduling and waits for any active poll to exit.
Other Intel drivers (ixgbe, ice, i40e) use napi_disable() without a
preceding napi_synchronize() in their down paths.

Remove redundant napi_synchronize() call.

Fixes: 2c6196013f84 ("igb: Add AF_XDP zero-copy Rx support")
Cc: stable@vger.kernel.org
Signed-off-by: Alex Dvoretsky <advoretsky@gmail.com>
---
Thanks for the suggestion, Maciej. I tested removing napi_synchronize()
and it fixes the issue cleanly -- napi_disable() handles the stuck poll
via NAPI_STATE_DISABLE without needing any hot-path changes.

v2:
  - Replaced 3-patch series with single napi_synchronize() removal,
    per Maciej Fijalkowski's suggestion. napi_disable() handles the
    stuck NAPI poll via NAPI_STATE_DISABLE, making the __IGB_DOWN
    checks in igb_clean_rx_irq_zc() and igb_tx_timeout(), and the
    transition guards in igb_xdp_setup(), all unnecessary.
  - Tested on Intel I210 (igb) with AF_XDP zero-copy: full E2E
    traffic suite, graceful shutdown, and 5x kill-9 stress cycles.
    Zero tx_timeout events.

 drivers/net/ethernet/intel/igb/igb_main.c | 1 -
 1 file changed, 1 deletion(-)
```

```
diff --git a/drivers/net/ethernet/intel/igb/igb_main.c
         b/drivers/net/ethernet/intel/igb/igb_main.c
index 12e8e30d8a2d..a1b3c5e4f7d2 100644
--- a/drivers/net/ethernet/intel/igb/igb_main.c
+++ b/drivers/net/ethernet/intel/igb/igb_main.c
@@ -2203,7 +2203,6 @@ void igb_down(struct igb_adapter *adapter)

         for (i = 0; i < adapter->num_q_vectors; i++) {
                 if (adapter->q_vector[i]) {
-                        napi_synchronize(&adapter->q_vector[i]->napi);
                         igb_set_queue_napi(adapter, i, NULL);
                         napi_disable(&adapter->q_vector[i]->napi);
                 }
```

Appendix B

How to Apply and Test the Patch

B.1 Prerequisites

- A Linux kernel source tree (v6.1 or later).
- An Intel I210 or I211 network controller using the `igb` driver.
- An AF_XDP-capable test tool such as `xdpsock` (from the kernel's `samples/bpf/` directory) or `xdp-tools`.
- A simple XDP program (e.g., `xdp_pass`).

B.2 Applying the Patch

Navigate to the kernel source directory and apply:

```
$ cd ~/linux
$ git am ~/patches/0001-igb-remove-napi_synchronize-in-igb_down.patch
```

B.3 Building and Installing

Build only the `igb` module:

```
$ make -C /lib/modules/$(uname -r)/build \
    M=drivers/net/ethernet/intel/igb modules
$ sudo modprobe -r igb
$ sudo insmod drivers/net/ethernet/intel/igb/igb.ko
```

Or rebuild and install the full kernel:

```
$ make -j$(nproc)
$ sudo make modules_install install
$ sudo reboot
```

B.4 Testing Procedure

B.4.1 Test 1: Kill AF_XDP Application

```
# Load XDP program
$ sudo ip link set dev enp8s0 xdp obj xdp_pass.o sec xdp

# Start AF_XDP zero-copy application
$ sudo xdpsock -i enp8s0 -q 0 -z &
$ AF_XDP_PID=$!

# Kill the application
$ sudo kill -9 $AF_XDP_PID

# Verify: no TX stall, interface recovers
$ ping -c 3 <gateway_ip>
$ dmesg | grep -i "watchdog\|timeout\|stall"
```

B.4.2 Test 2: Remove XDP Program While AF_XDP Is Running

```
$ sudo ip link set dev enp8s0 xdp obj xdp_pass.o sec xdp
$ sudo xdpsock -i enp8s0 -q 0 -z &

# Remove XDP program while application is running
$ sudo ip link set dev enp8s0 xdp off

# Verify: clean recovery
$ ping -c 3 <gateway_ip>
```

B.4.3 Test 3: Kill-9 Stress Test (5 Cycles)

```
for i in $(seq 1 5); do
    echo "=== Stress cycle $i ==="
    sudo ip link set dev enp8s0 xdp obj xdp_pass.o sec xdp
    sudo xdpsock -i enp8s0 -q 0 -z &
    PID=$!
    sleep 0.5
    sudo kill -9 $PID
    wait $PID 2>/dev/null
    sudo ip link set dev enp8s0 xdp off 2>/dev/null
    sleep 0.5
done

# Verify: interface stable after all iterations
$ ping -c 3 <gateway_ip>
$ ethtool -S enp8s0 | grep tx_timeout
```

B.5 Expected Results

Test	Before Patch	After Patch
Kill AF_XDP app	TX stall, reboot needed	Clean recovery
Remove XDP while ZC	TX stall or oops	Clean recovery
5x kill-9 stress cycles	Consistent stalls	Zero tx_timeout events

Appendix C

How to Submit Patches to the Linux Kernel Mailing List

C.1 Formatting

Linux kernel patches must follow a strict format. The `git format-patch` command generates properly formatted patches:

```
$ git format-patch -1 -v2 -o outgoing/
```

This creates a single patch file: `v2-0001-igb-remove-napi_synchronize-in-igb_down.patch`. For a multi-patch series, use `-N -cover-letter` where N is the number of commits.

C.2 Running `checkpatch.pl`

Before submitting, verify that the patches pass the kernel's style checker:

```
$ ./scripts/checkpatch.pl outgoing/*.patch
```

Fix any warnings or errors. The most common issues are:

- Lines exceeding 80 characters (now 100 in some subsystems).
- Missing `Signed-off-by` line.
- Incorrect indentation (tabs, not spaces, for C code).

C.3 Identifying the Maintainers

Use `get_maintainer.pl` to find the correct mailing lists and maintainers:

```
$ ./scripts/get_maintainer.pl outgoing/v2-0001-*.patch
Tony Nguyen <anthony.l.nguyen@intel.com> (maintainer)
intel-wired-lan@lists.osuosl.org (list)
netdev@vger.kernel.org (list)
```

C.4 Sending with `git send-email`

```
$ git send-email \
    --to intel-wired-lan@lists.osuosl.org \
    --cc netdev@vger.kernel.org \
    --cc maciej.fijalkowski@intel.com \
    --cc aleksandr.loktionov@intel.com \
    --cc anthony.l.nguyen@intel.com \
    --in-reply-to='<original-v1-message-id>' \
    --thread \
    outgoing/v2-0001-*.patch
```

Key options:

- `-to`: primary mailing list (subsystem list).
- `-cc`: secondary lists, maintainers, and prior reviewers.
- `-in-reply-to`: threads v2 as a reply to the v1 discussion.
- `-thread`: ensures proper email threading.

C.5 The Review Process

After submitting:

1. Wait for automated CI bots (e.g., `patchwork`, kernel test robot) to report build/test results.
2. Maintainers and reviewers will reply on the mailing list with comments, questions, or `Reviewed-by`/`Acked-by` tags.
3. Address feedback by sending a `v3` series with `git format-patch -v3`.
4. Once accepted, the patches flow through the maintainer's tree (e.g., `intel-wired-lan/net`) into `net.git`, and eventually into Linus's tree.

C.6 Backporting to Stable

The `Cc: stable@vger.kernel.org` tag in the commit message signals that the patch should be backported to stable kernel releases. The stable team will pick it up automatically after it lands in Linus's tree.